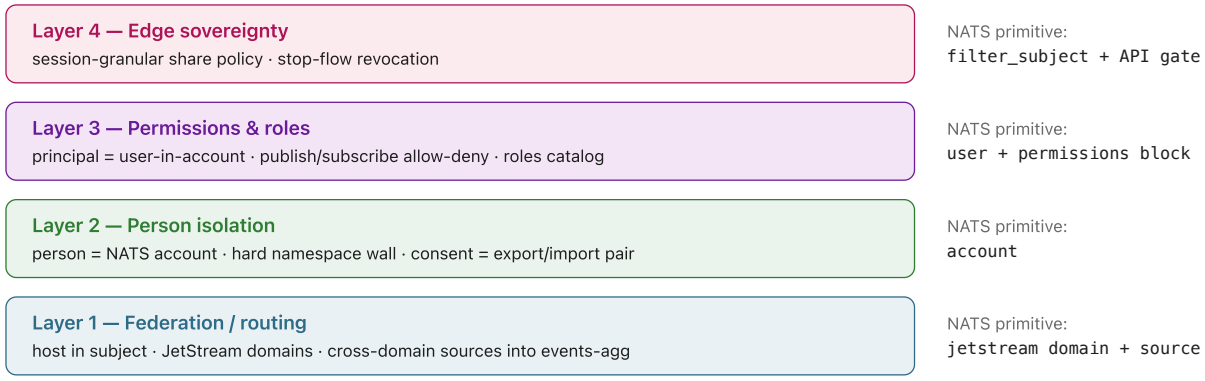
Identity & permissions — architecture field report

As built on `feat/federation-host-in-subject` (PR #58, which absorbed PR #54) — verified by booting the full stack live on 2026-06-11. Every claim below was either exercised tonight or cites the file that implements it.

1 The four-layer model

Each layer maps a sovereignty concern onto one native NATS primitive. Nothing is invented; everything is configuration of the bus.



Layer	Status on this branch	Where
1 — Federation	complete — T1/T2/T3 converge in the lab; T2 verified live tonight	`rs/bus/src/nats_bus.rs`, `rs/tests/test_federation_lab.rs`
2 — Person isolation	shipped — accounts generated from Person records, isolation proven by testcontainers	`rs/bus/src/accounts.rs`, `rs/bus/tests/test_multi_account_isolation.rs`
3 — Permissions / roles	shipped — roles directory, role-gated writes, per-user profiles	`rs/server/src/directory.rs`, `rs/server/src/account_config.rs`

4 — Edge sovereignty	partial — API gate on 2 endpoints + publish-time skip; `filter_subject` reconciliation deferred	`rs/bus/src/share_policy.rs`, `rs/server/src/admin.rs`

2 Identity: Person and Principal

A Person is a human. A Principal is one (host, user) seat in that human's fleet. The split is the whole design.

Identity is configured, bootstrapped, and then *stamped onto every event*:

1. **Config:** a `[person]` block in `config.toml` declares the Person (UUID) and one or more Principals, each with matchers (`host`, `user`, optional agent / watch-dir pattern). First boot auto-creates it from the detected `(host, user)`.
2. **Stamping:** `principal_resolver.rs` is a pure function from a source context to `(person_id, principal_id)`; the watcher stamps every translated batch, so every CloudEvent carries both as extension fields (`rs/core/src/cloud_event.rs`).
3. **Persistence:** sessions store both IDs on SQLite and Mongo with conformance parity; `/api/sessions` exposes them; `/api/fleet` serves the configured fleet; the sidebar groups sessions by principal.

The mapping onto the bus is then mechanical: **Person → NATS account, Principal → user inside that account.** Identity stops being an application-level filter and becomes a property of the wire.

3 The permissions substrate: one account per person

Generated, never hand-written. The conf below is the one actually running tonight.

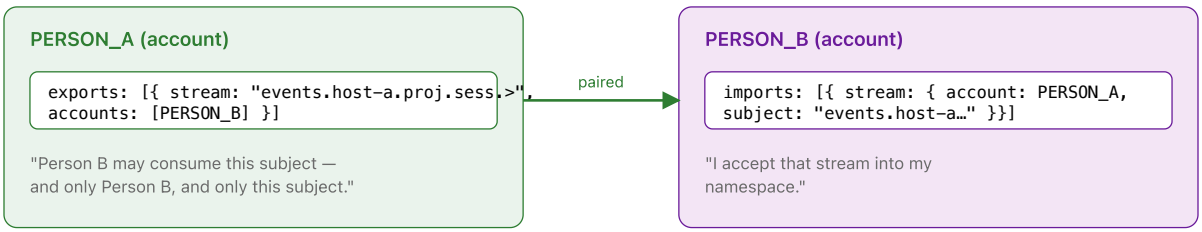
```
# data/nats-accounts.conf — generated by AccountConfigWriter
listen: 0.0.0.0:4222
jetstream {
  store_dir: "./data/nats-jetstream"   # quoted — field fix #1
}
accounts {
  PERSON_29911D8F_3893_45D3_A22D_E66073C03A6F: {
    jetstream: enabled                 # per-account opt-in — field fix #3
    users: [
      { user: "29911d8f-…", password: "29911d8f-…-local-dev" }
```

```
    ]
  }
}
```

- **Generation:** `render_accounts_block()` (`rs/bus/src/accounts.rs`) is a pure function from `AccountSpec`s to conf text. `AccountConfigWriter` (`rs/server/src/account_config.rs`) persists it at boot and rewrites it on every share mutation, then reloads NATS via SIGHUP.
- **Isolation is absolute by default:** an account is a namespace wall. Another person's user cannot subscribe to your subjects, query your streams, or even learn they exist. The testcontainer suite (`test_multi_account_isolation.rs`) proves messages do not leak between two person accounts on a shared server.
- **Credentials:** deterministic `{person_id}-local-dev` passwords for local dev; the Phase 6.7 TODO swaps these for NKEYs. The server itself authenticates as the person's user — its `nats_url` carries `user:pass@`, which the bus now extracts into `ConnectOptions` (field fix #2).

4 Consent is a paired export/import

Either side alone delivers nothing. That asymmetry is the security model.



The operator gesture is `POST /api/admin/share-with-person`: `add_share_with_stubs()` writes the matched export/import pair (creating a stub account for the target person if needed), persists the conf, and SIGHUPs NATS. The end-to-end is proven live by the Phase 5.10 test: events published in one account arrive in the other *only after* the reload — and nothing else leaks.

5 Roles and per-user permission profiles

Layer 3 narrows what a principal may do inside its own account.

- **Roles directory** (`rs/server/src/directory.rs`): persisted person→role assignments with a grant-role bootstrap; surfaced in the Participants admin panel.
- **Role-gated writes:** share-policy mutations require the operator role; `admin_token` is a separate tier from `api_token`, so observation and administration are different credentials.
- **Per-user profiles** (`render_permission_field()`): each user can carry `publish` / `subscribe` allow-lists. The deliberate subtlety: an *empty* allow-list renders as `{ deny: [">"] }` — NATS shorthand would have read it as "no restrictions", the exact opposite. An Observer role therefore *provably cannot publish*, enforced by the bus, not by application politeness.

6 Share policy: the sovereignty invariants

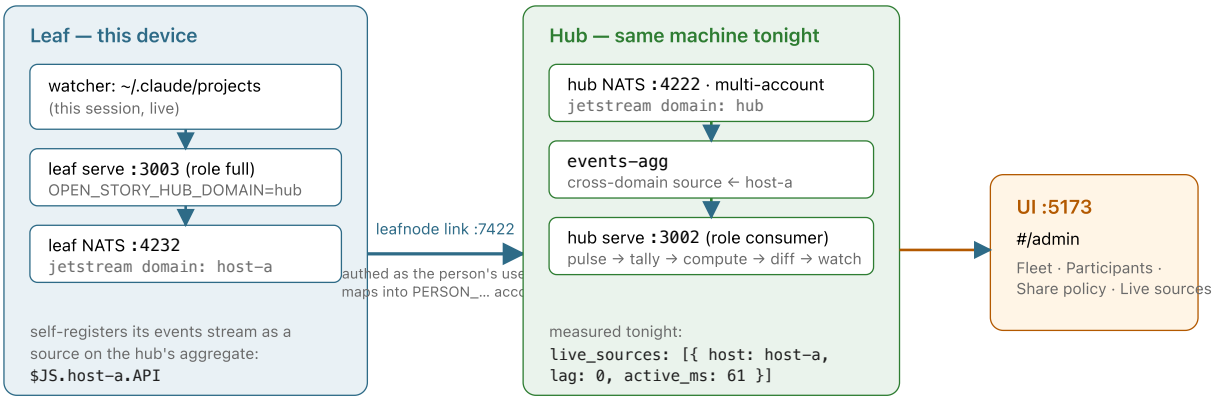
Default `shared`; turning something private is the explicit act. Three invariants, each named and tested.

Invariant	Meaning	Enforced at
① Catch-up respects share	Private sessions vanish from `/api/digests`; per-session event reads return 404 — existence is denied so peers stop asking. Publish-time enforcement also gates the bus: `NatsBus.publish` skips private sessions (`share_policy.rs`).	API + bus
② Retention keeps own	`cleanup_old_sessions(days, keep_host)` never deletes sessions whose host is this device, regardless of age.	store
③ Revocation = stop-flow, not purge	Marking private gates new visibility immediately and preserves local events; peer-mirrored copies persist. This is a *named hard edge*, not an oversight — cooperative purge across consent boundaries is future work.	API (by design)

> **Honest gap (from the PR's own review):** only 2 of ~14 per-session read endpoints consult the policy today. Federation propagation of a private session is correctly blocked, but a client with API access can still read "private" session content through e.g. `/transcript` or `/records`. Until the shared gate lands, UI copy should claim "hidden from peer catch-up", not "private".

7 The transport underneath: T2 as verified tonight

Everything above rides this. One machine, two NATS domains, a real leafnode link — and this very report's authoring session flowing through it.



The admin surface is fully push-driven: bus events pulse the `admin_broadcaster` actor, which re-tallies, recomputes the topology pure-functionally, diffs against the cached frame, and only publishes changes into a `tokio::watch` — the server-side twin of an RxJS `shareReplay(1)`. REST borrows the current frame; WebSocket subscribers await changes. The roster distinguishes provenance honestly: a node can be present from *stored session evidence* (`source: "sessions"` — e.g. the `a1` Linux box, 14 historical sessions, currently dark) without ever appearing in *live sources*, which only reports what JetStream can prove right now.

8 Field findings — four bugs between "all tests green" and "it boots"

The branch's permissions logic was well-tested and correct; the production wiring had never been run end-to-end. One evening of dogfooding found what the suites could not.

#	Symptom	Root cause	Fix
1	NATS refuses to boot: `Parse error: 'Floats must start with a digit'`	Generated conf wrote `store_dir: ./data/…` unquoted; the NATS parser lexes a leading `.` as a float	`de7cd1f` — quote the path + pinning test
2	Server can't connect to its own multi-account bus: `authentication error`	async-nats ignores URL userinfo; the bus only special-cased token auth, so `user:pass@` URLs connected anonymously. The isolation tests knew (a comment says	`99d3a6b` — `extract_user_pass()` → `ConnectOptions` + tests

		so) and worked around it — in the tests only	
3	Connect succeeds, then `failed to create/get 'events' JetStream stream`	Explicitly-defined NATS accounts default JetStream *off*; the generator never emitted the per-account opt-in	`82ba304` — emit `jetstream: enabled` per account (red→green)
4	Bus provably federating (lag 0) while the Admin panel swears "Not federated"	The pulse forwarder's `while let Ok(…) = recv()` exits on the first `Lagged` error — the opposite of its own comment. A leaf backfill burst laps the ring once and the topology frame freezes forever	`f1c05db` — extracted `spawn_pulse_forwarder()`: survive Lagged, coalesce via `try_send`; test overflows the ring deliberately

> **Also observed, not yet fixed:**
>
> • `AccountConfigWriter` rewrites the conf at every boot and on every share write, dropping hand-added `domain` / `leafnodes` blocks — the static prefix needs to learn federation config.
>
> • A fresh data dir bootstraps a fresh Person UUID (the demo leaf minted its own), so one human on two data dirs becomes two "persons". Identity needs to travel with the operator, not the directory.
>
> • The Share Policy table renders all ~560 sessions flat; the policy model is sparse (default + exceptions) and the UI should be too.

9 What is deliberately not here yet

- **Bus-level share enforcement via `filter_subject`:** needs the subject scheme to encode shared/private so JetStream can route on it (Phase 4 follow-up). Until then the API gate + publish-time skip are the enforcement.
- **NKEY credentials:** deterministic local-dev passwords are a placeholder by design; the conf shape is already NKEY-compatible.
- **Cooperative purge across consent boundaries:** revocation stops flow; it does not (and currently cannot) reach into a peer's store. Named as the hard edge of invariant ③.
- **Cross-machine T2 with this stack:** tonight's star ran on one machine; the Tailscale-distributed version needs the conf-clobbering fix and a leafnode listener on the tailnet

interface.
